Technische Universität Berlin

Distributed Systems
Chapter 1: Introduction

FG Verteilte Systeme und Betriebssysteme | Distributed Systems

Agenda

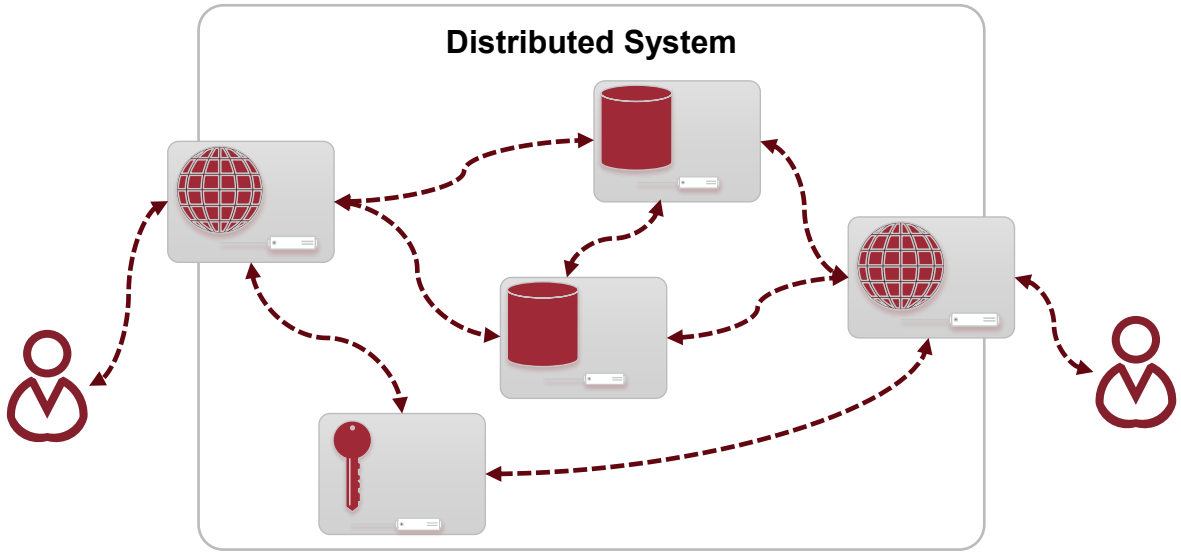
1.1 Characterization of Distributed Systems

1.2 Motivation

1.3 Challenges

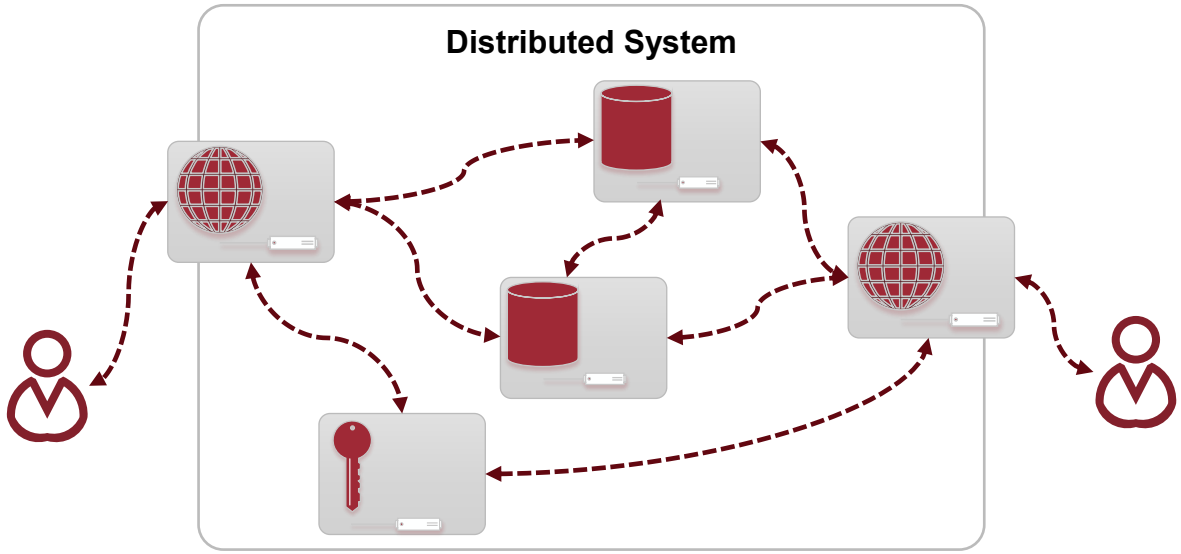
Characterization

Complex software systems consist of a large number of computers …



Characterization

Complex systems consist of a large number of HW / SW / AI components…

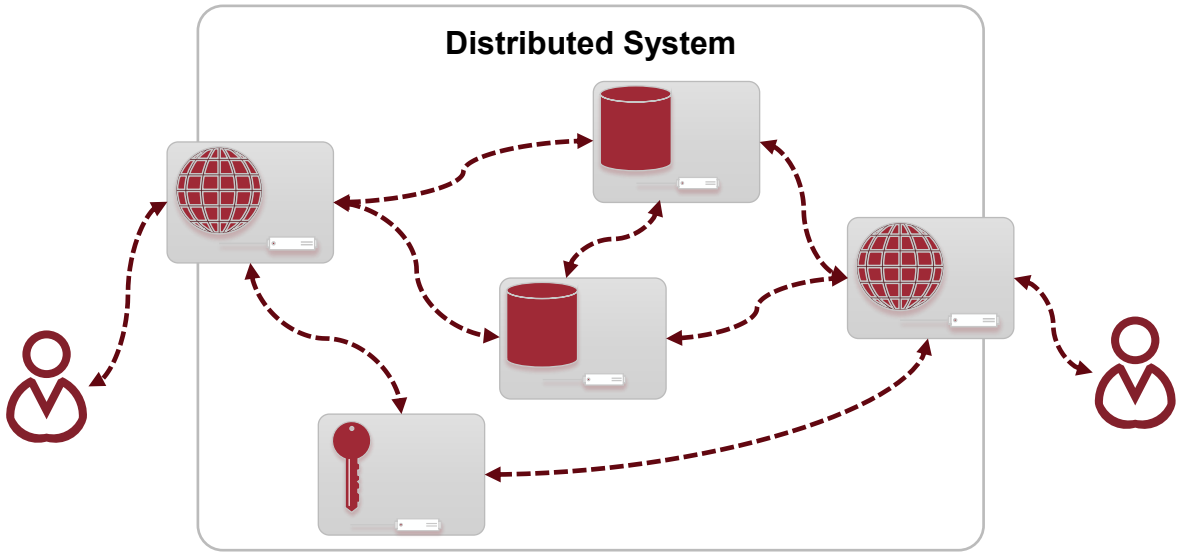


Basic Characteristics

- Large number of specialized agents, working at the same time
- Cooperation to fulfill complex tasks
- Competition for limited resources

Characterization

Perception: group of autonomous „intelligent" agents, which interact with each other and with the outside world



Definition

Definition by Coulouris:

- „A distributed system is one in which hardware or software components are located at networked computers and communicate and coordinate their actions only by passing messages."

Definition by Tanenbaum:

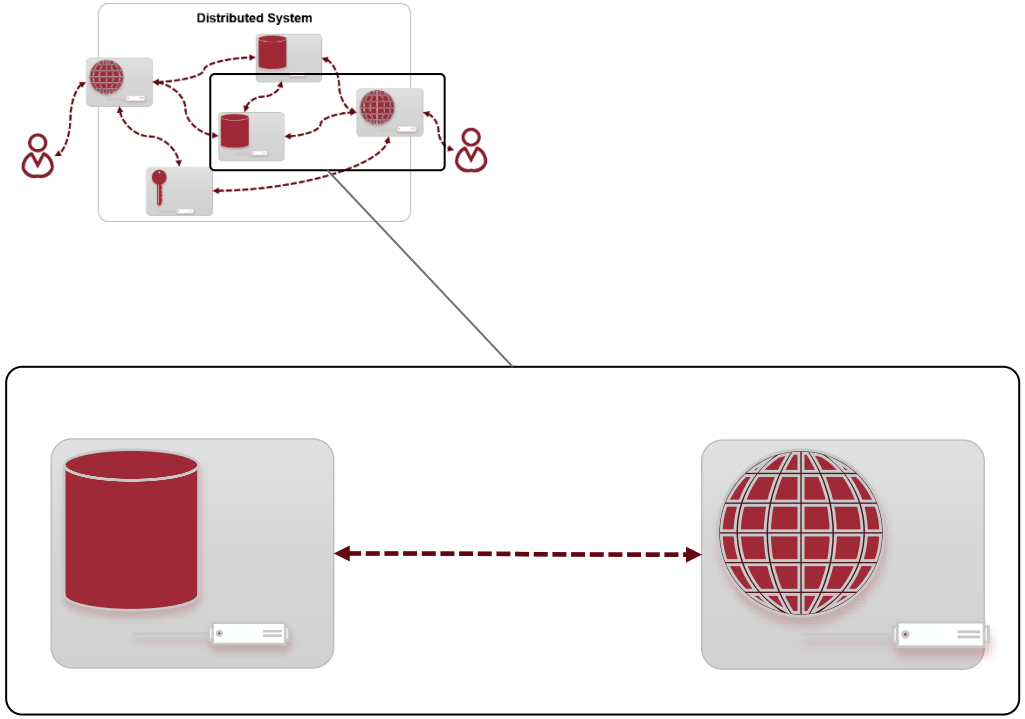
- „ A distributed system is a collection of independent computers that appears to its users as a single coherent system."

Definition by Lamport (;-)):

- „ A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable."

Definition (2)

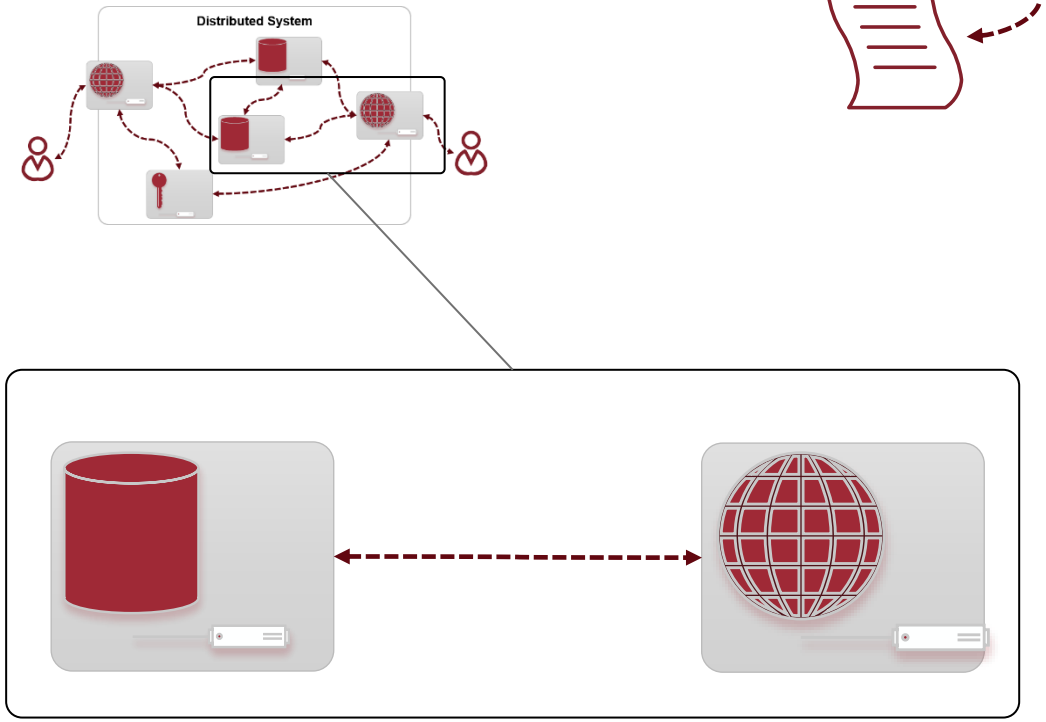
A cascading approach:



Source:
http://www.hpcs.cs.tsukuba.ac.jp/~tatebe/lecture/h23/dsys/dsd-tutorial.html

Definition (2)

A cascading approach:

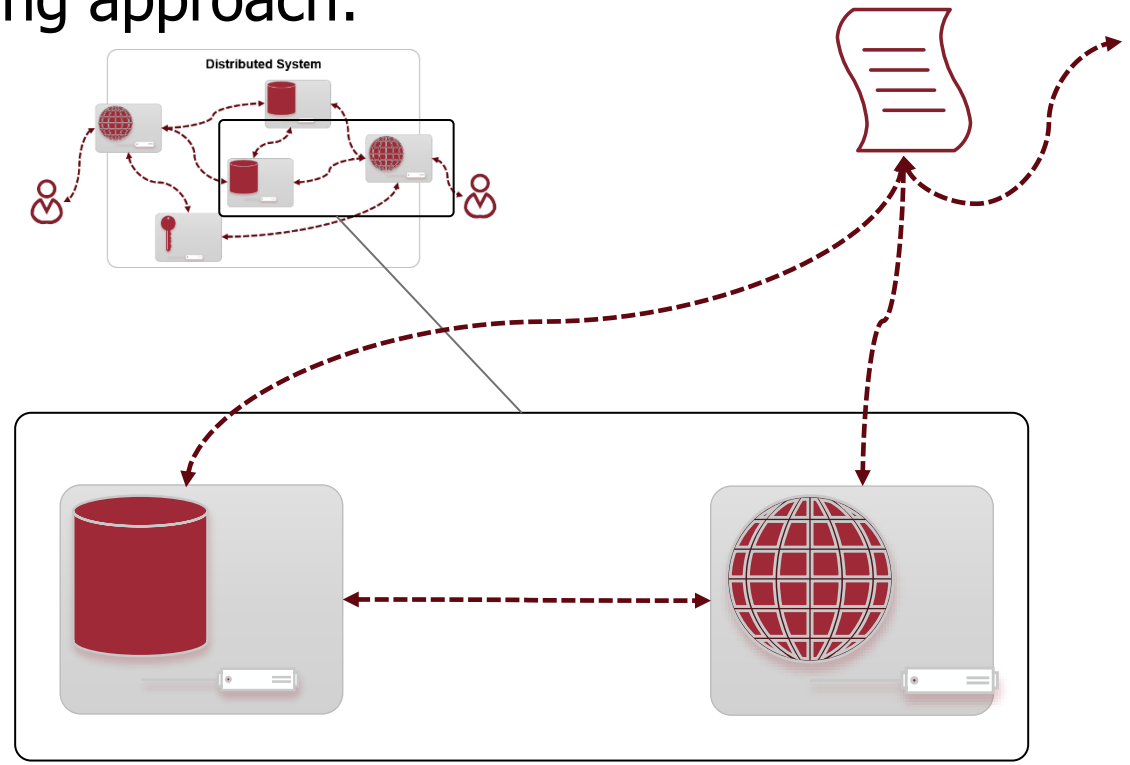


A program

– is the code you write.

Definition (2)

A cascading approach:



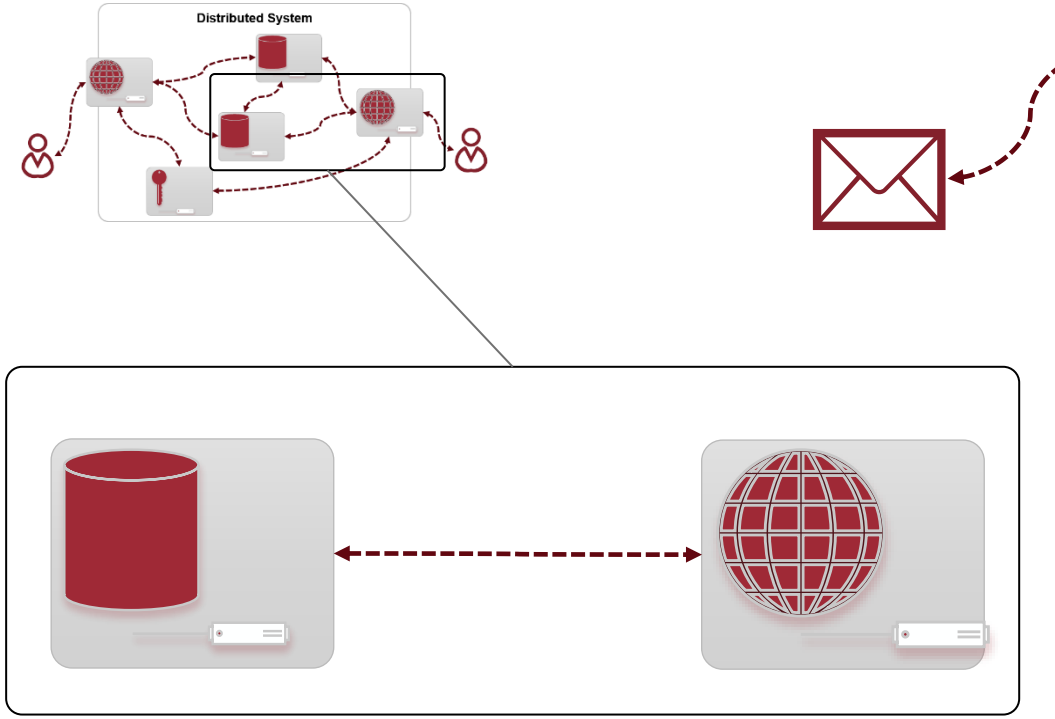
A program

A process

– is what you get when you run it.

Definition (2)

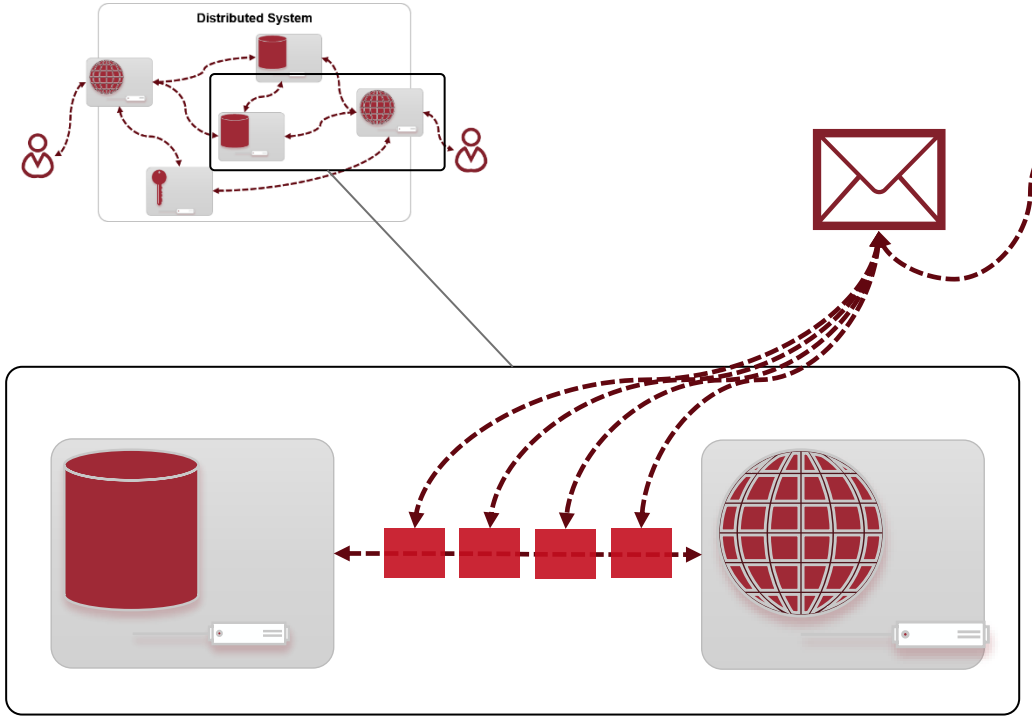
A cascading approach:



A program
A process
A message
– is used to communicate between processes.

Definition (2)

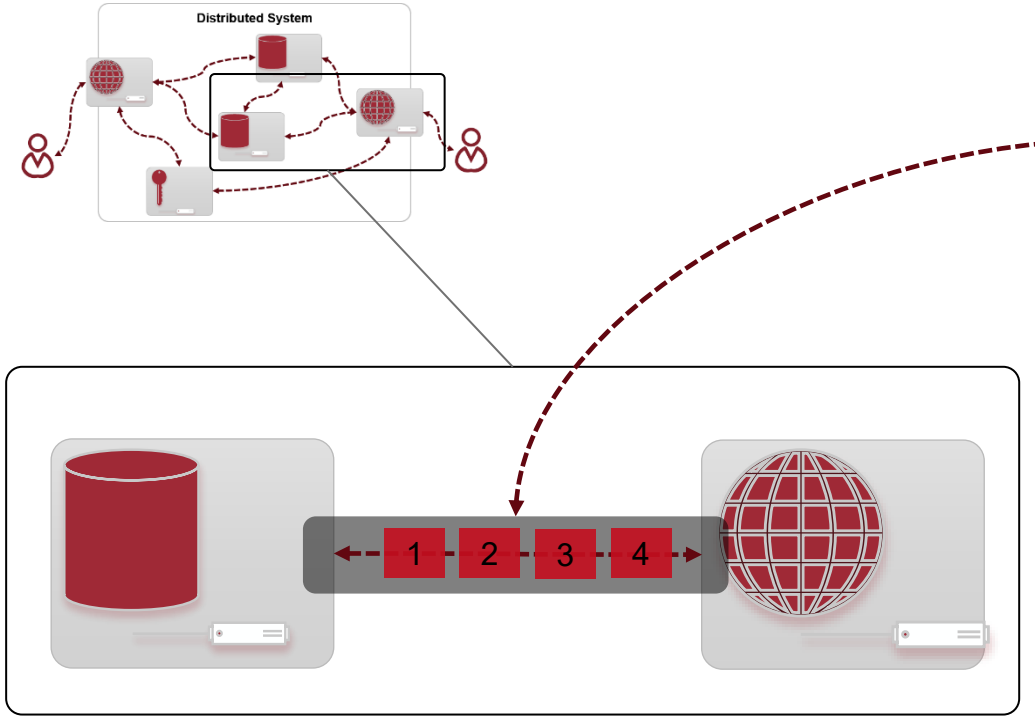
A cascading approach:



A program
A process
A message
A packet
– is a fragment of a message that might travel on a wire.

Definition (2)

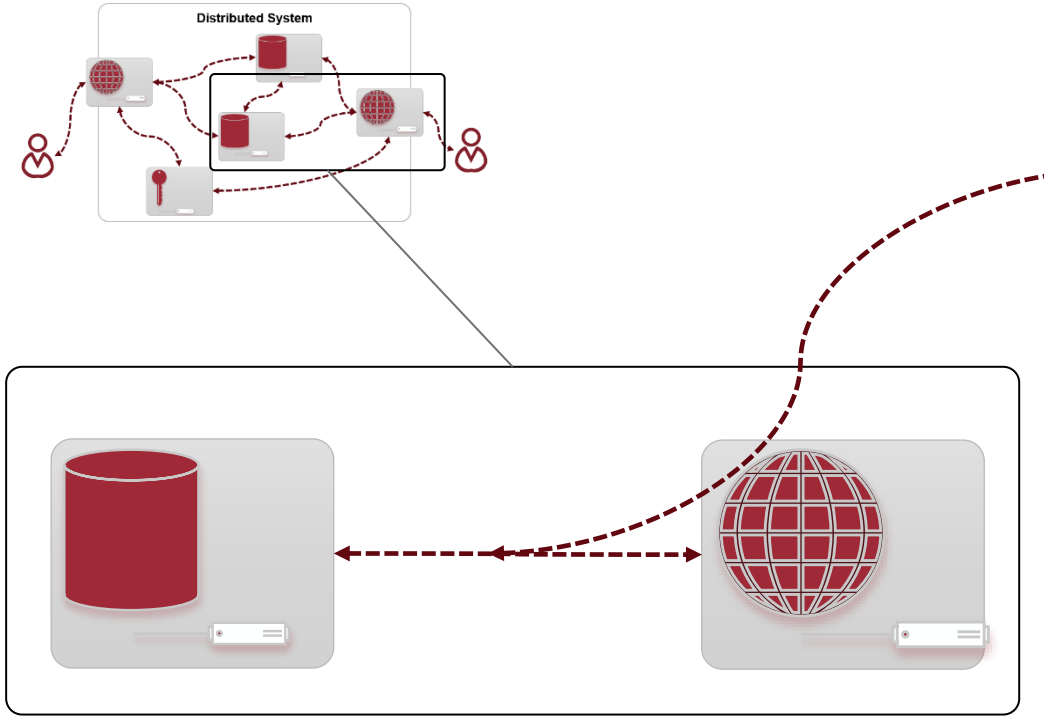
A cascading approach:



A program
A process
A message
A packet
A protocol
– is a formal description of message formats and the rules that two processes must follow in order to exchange those messages.

Definition (2)

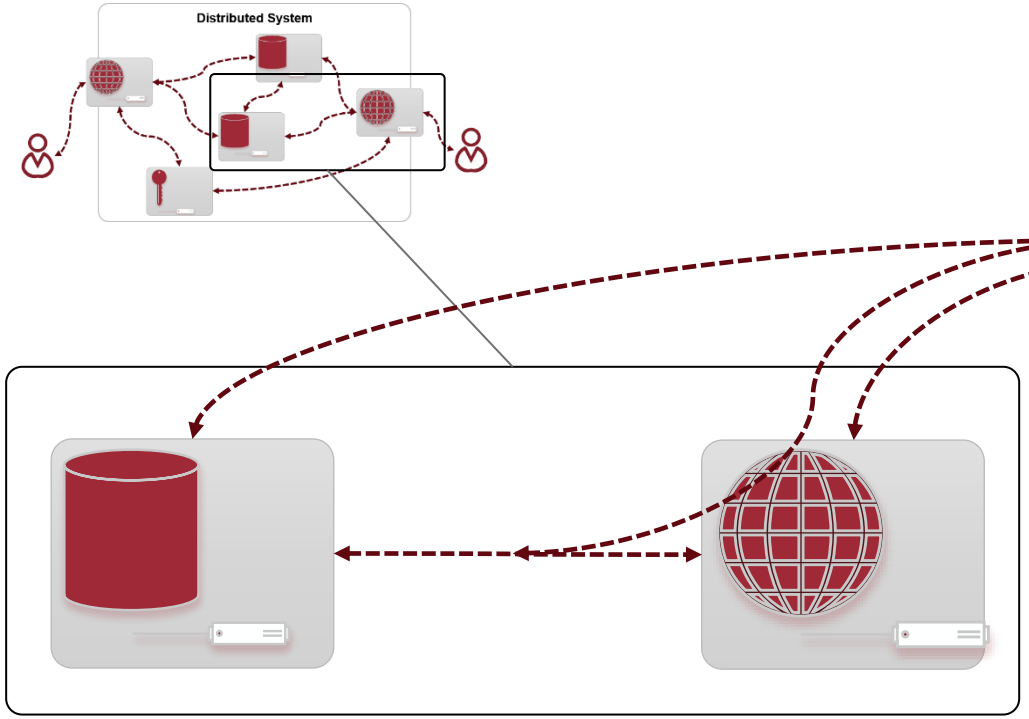
A cascading approach:



A program
A process
A message
A packet
A protocol
A network
- is the infrastructure that links computers, workstations, terminals, servers, etc. It consists of routers which are connected by communication links.

Definition (2)

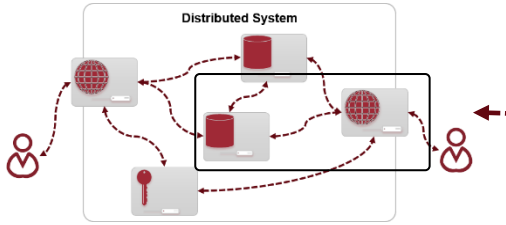
A cascading approach:



A program
A process
A message
A packet
A protocol
A network
A component
- can be a process or any piece of hardware required to run a process, support communications between processes, store data, etc.

Definition (2)

A cascading approach:



A program
A process
A message
A packet
A protocol
A network
A component

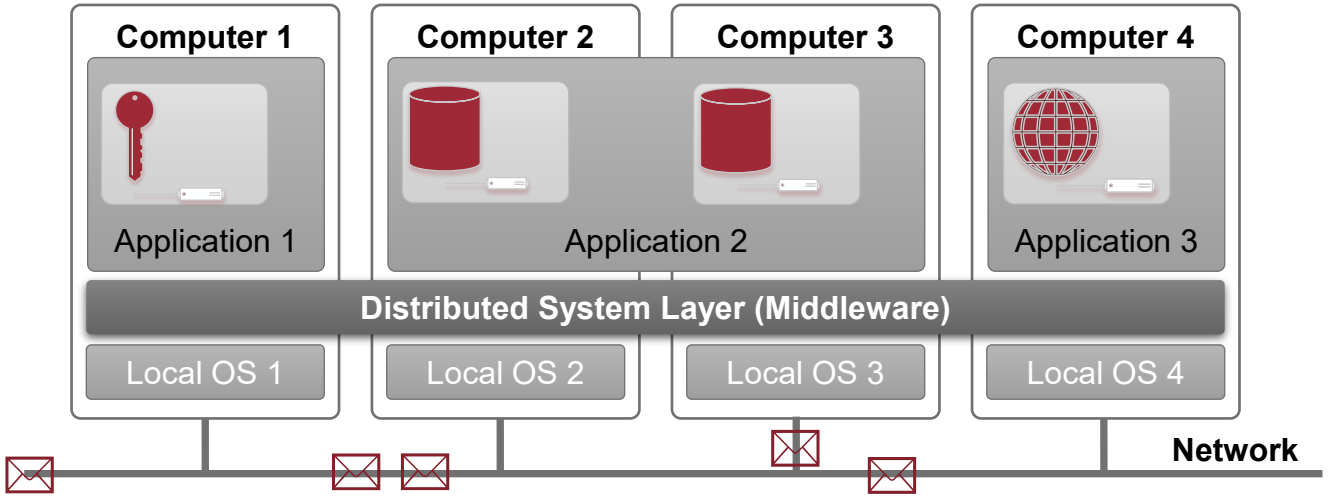
A distributed system

executes a collection of protocols to coordinate the actions of multiple processes on a network, such that all components cooperate to perform a single or small set of related tasks.

Definition - Summary

A distributed system: collection of independent computers that

- Are connected through a network and
- Appear to be one connected computing system and
- Communicate and coordinate their actions only by passing messages

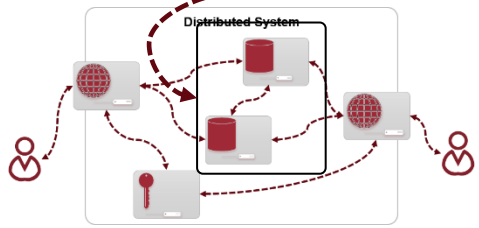


Basic Properties

Concurrent activities:

– Coordination and synchronization required

Bank account example
Replicated Databases



Alice account contains 1000€.

Bob, the account manager updates account with 1% interest

Alice, the customer deposits 100€ to her account

DB 1: 1110€ account value
DB 2: 1111€ account value

Basic Properties

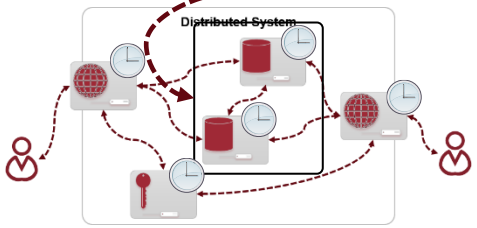
Concurrent activities:

No global clock:

- Problem of timing due to limited possibilities for local clock synchronization

Bank account example

Replicated Databases



- Each node has its own local clock
- Local clocks drift
- **What event happened first?**

Basic Properties

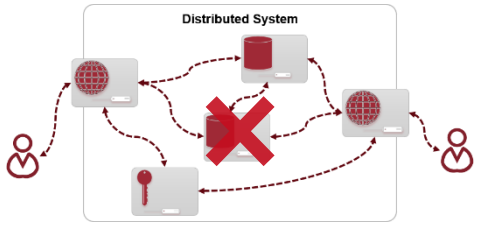
Concurrent activities:

No global clock:

High probability of errors and failures:

- Very large systems / applications, interaction by passing messages
- Failures in local computing nodes, network, or communication procedure
- Fault Tolerance is necessary

Bank account example
The law of conservation of money



The servers of Bob‘s bank suddenly crash

Alice, transfers 100€ to Bob‘s account

100€ withdrawn and disappeared!

Basic Properties

Concurrent activities:

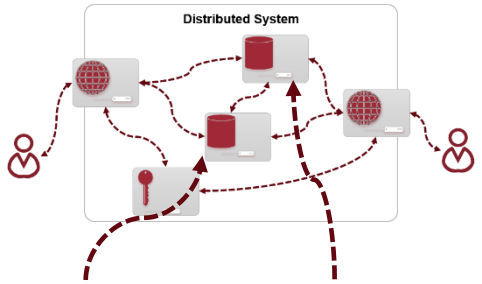
No global clock:

High probability of errors and failures:

Heterogeneous hardware and software components:

- Standardization of interfaces is required

Bank account example
Heterogeneity of systems



- Big Endian
- HP-UX OS
- C based Application

- Little Endian
- Windows OS
- Java based Application

Introduction

1.2 Motivation

Motivation

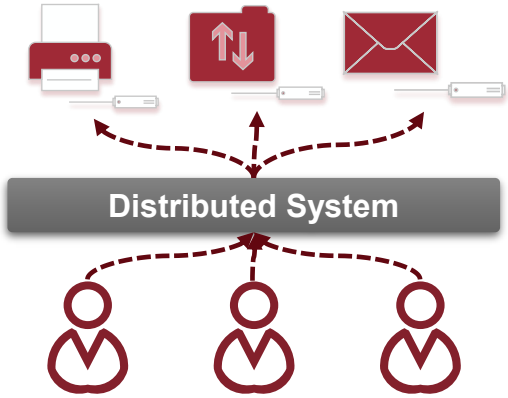
Design, implementation and operation of distributed systems is obviously more complex than with traditional centralized or monolithic systems.

Why invest time and money in building a distributed system?

Motivation

Reason 1: Collaboration, efficiency and utilization through resource sharing

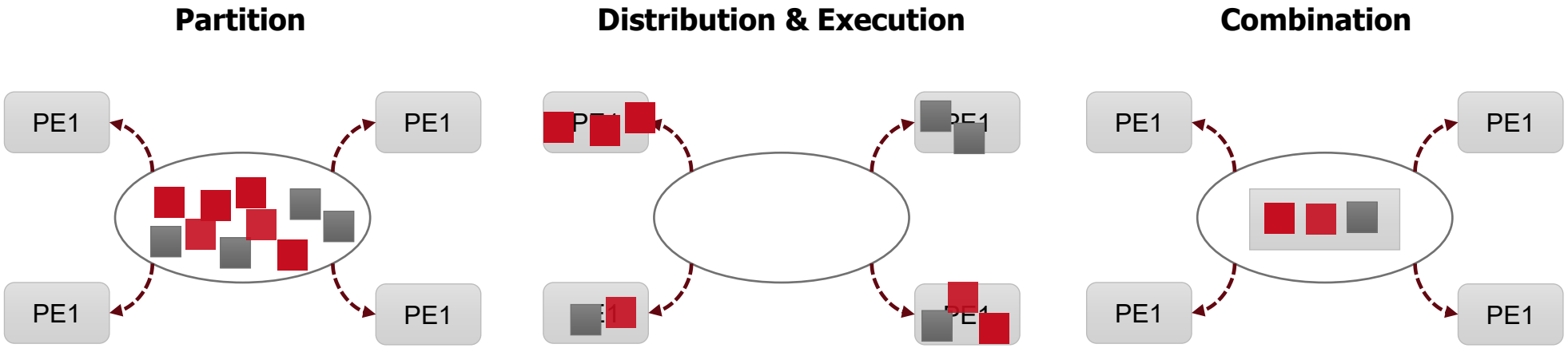
- Resource: hard- and software components that can be used together in a computer network
- Examples:
  - Shared printing, hard disks, ...
  - Shared databases, calendars, files, …
  - Shared services like web servers, e-mail, ...



Motivation

Reason 2: Speeding up of processing

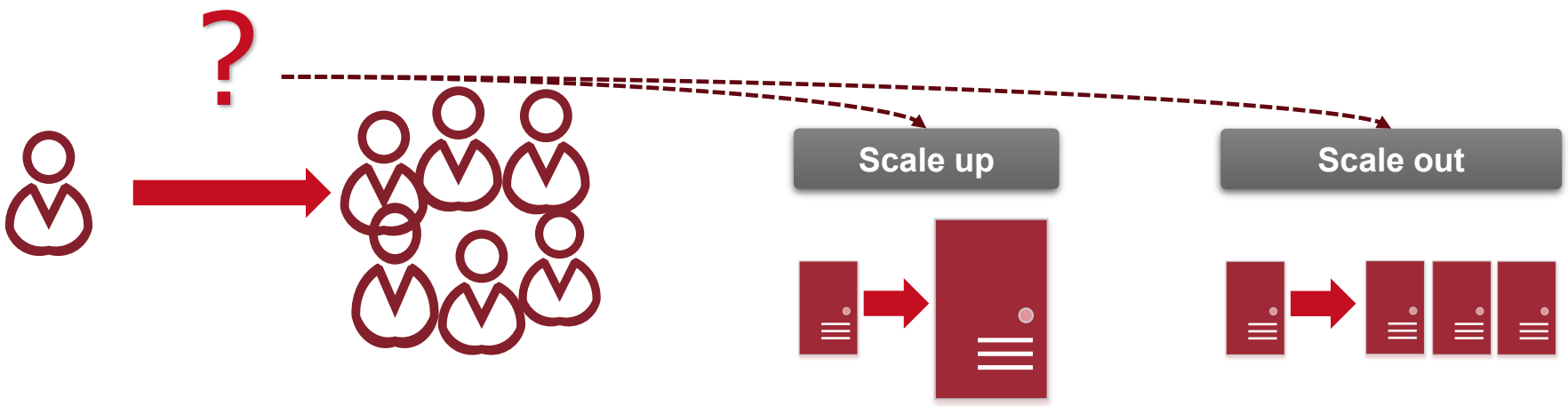
– Use of independent connected processing elements (PE) for simultaneous work on independent sub-problems



Motivation

Reason 3: Scalability (Serve one million customers with a single server?)

- Monolithic / centralized systems difficult to scale
- Scale vertically (scale up) vs. scale horizontally (scale out)
- Scale up very limited, while scale out only works with a distributed system



Motivation

Reason 1: Increasing collaboration, efficiency and utilization through resource sharing

Reason 2: Speeding up of processing

Reason 3: Scalability

Reason 4: System stability and availability because of redundancies

- Redundant data storage on computers located in different places
- Redundant execution of computing operations to trap failures of single servers, …

Motivation

Reason 1: Increasing collaboration, efficiency and utilization through resource sharing

Reason 2: Speeding up of processing

Reason 3: Scalability

Reason 4: System stability and availability because of redundancies

Reason 5: Modeling real situations, characterized by separation of their components

- Connected services and clients (Internet, ATMs)
- Independent connected units in cars, factories, companies, household...
- Creation of company networks
- Economics

Pitfalls

The common pitfalls or false assumptions when developing a distributed application for the first time (by Peter Deutsch / James Gosling):

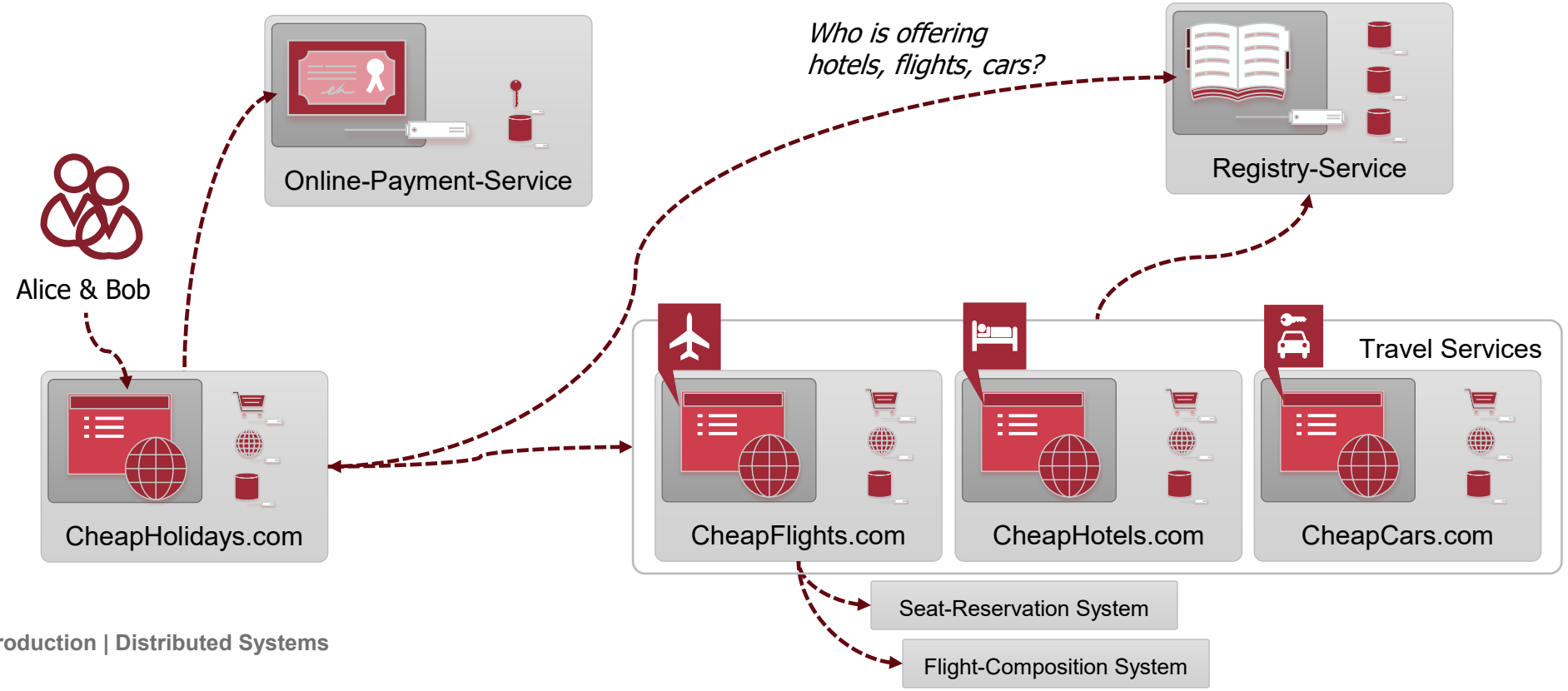
- The network is reliable.
- The network is secure.
- The network is homogeneous.
- The topology does not change.
- Latency is zero.
- Bandwidth is infinite.
- Transport cost is zero.
- There is one administrator.

Introduction

1.3 Challenges

The Guiding Thread Example …

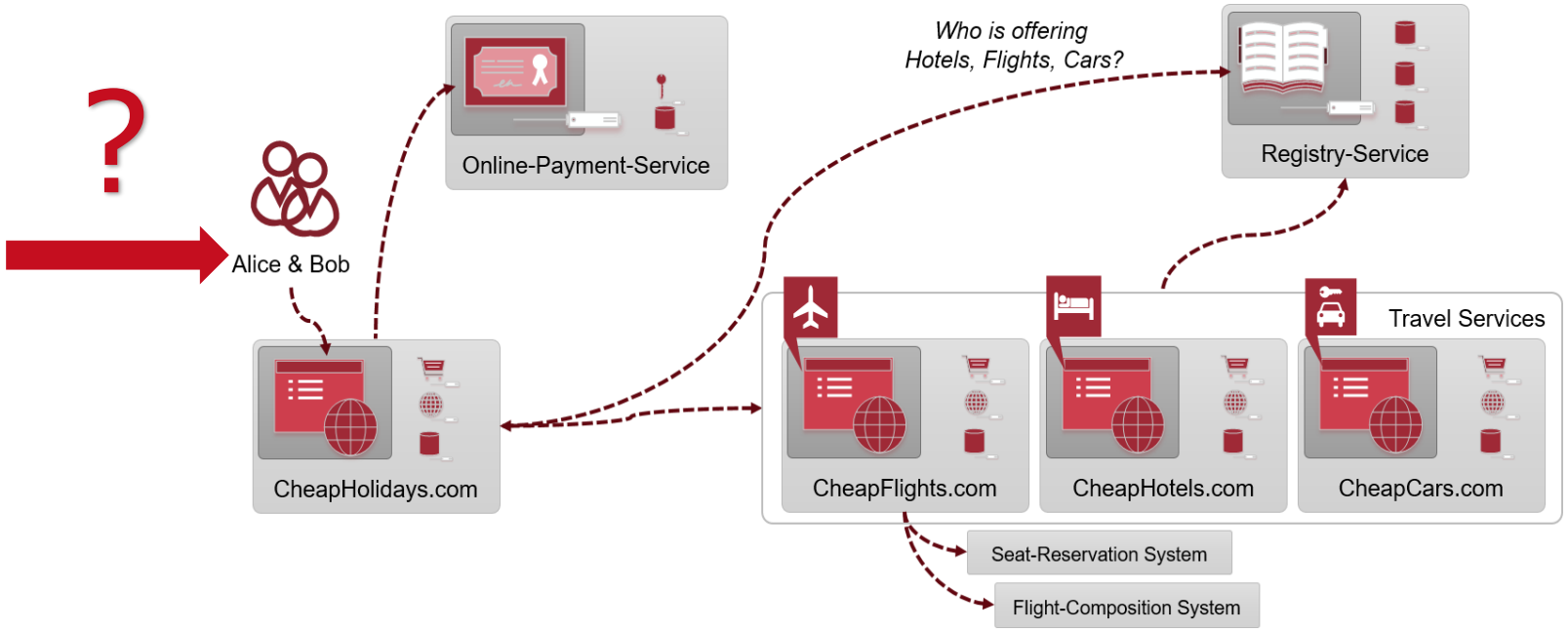
Online Travel Agency Service (simplified):



Challenges …

… we have to face when developing a distributed system:

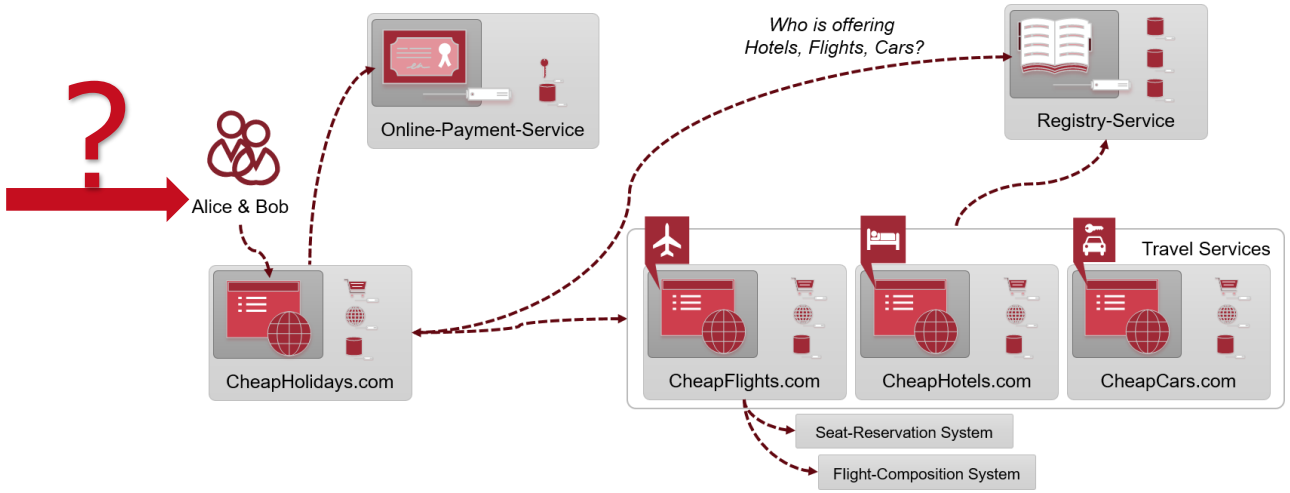
- Heterogeneity
- Openness
- Security
- Scalability
- Failure Handling
- Concurrency
- Transparency
- Quality of Service



Challenges …

… we have to face when developing a distributed system:

- **Heterogeneity**
- Openness
- Security
- Scalability
- Failure Handling
- Concurrency
- Transparency
- Quality of Service



CheapFlights.com uses Windows, CheapHotels.com uses Linux-based web servers.

CheapCars.com plans to migrate its interfaces from web services to a REST-based approach.

Challenge Heterogeneity

Distributed systems (DS) based on different networks, computers, OS, programming languages, implementations, … → DS components are typically heterogeneous!

DS composed of heterogeneous systems requires standards for:

- Shared network protocols
- Interchangeable formats of data/data structures, independent of hardware
- Inter-process-communication (e.g. exchange of messages, database queries, …)

Approaches for solving heterogeneity (i.e. achieving interoperability):

- Middleware: software layer providing a programming abstraction and masking the heterogeneity of underlying components
- Virtual Machines: compiler generates code for a process-level virtual machine, not for target hardware (e.g. Java Virtual Machine), as well as system virtualization

Challenge Heterogeneity

Heterogeneity is considered obstacle, but:

- Allows efficiently designing a system towards the specific needs of target environment
- Heterogeneity raises the importance for interoperability

IEEE definition of interoperability: The ability of two or more systems or components to exchange information and to use the information that has been exchanged.

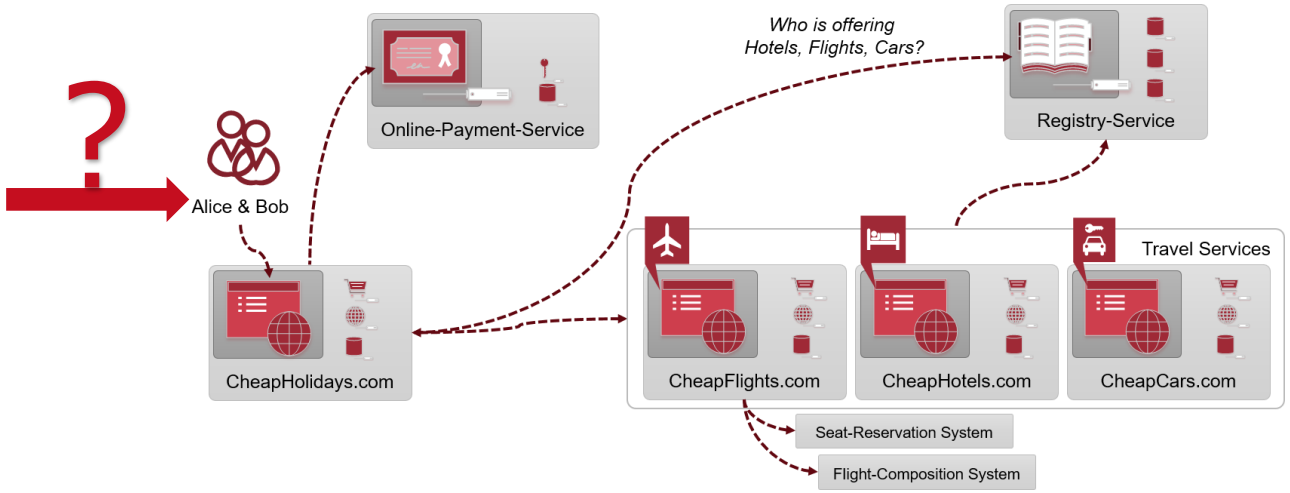
We know different levels (skipping level 0):

- Level 1: Technical Interoperability - A communication infrastructure exists
- Level 2: Syntactic Interoperability - The structure of the data is defined
- Level 3: Semantic Interoperability - The meaning of the data is defined

Challenges …

… we have to face when developing a distributed system:

- Heterogeneity
- **Openness**
- Security
- Scalability
- Failure Handling
- Concurrency
- Transparency
- Quality of Service



The competitor CheaperFlights.com wants to offer its services as well and must be integrated. No problem due to the use of a common messaging protocol and standardized interfaces.

Challenge Openness

Openness is the extensibility of the system

- Degree up to which new services can be added and provided for use by different clients
- Publication of specification / documentation of interfaces and components. Examples:
  - Internet protocols: RFC-Documents (Requests For Comment), consecutive numbering, include discussions and specifications (see www.ietf.org/rfc.html)
  - CORBA: a number of technical documents (see www.omg.org)

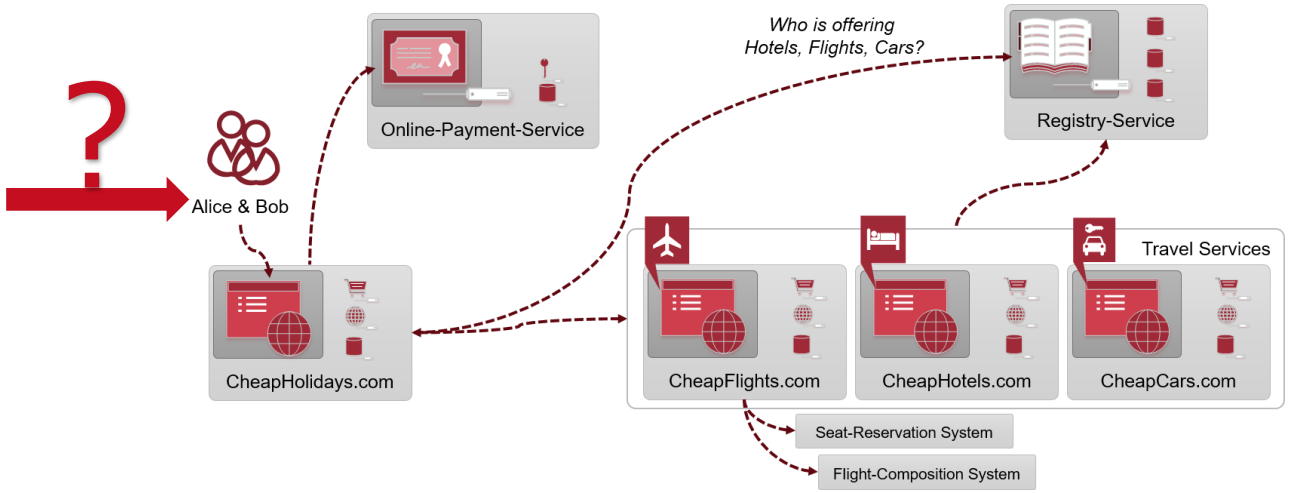
Characteristics of open distributed systems:

- Key technologies and interfaces are disclosed
- Uniform communication mechanisms exist
- Independent of vendor → can be implemented from heterogeneous HW/SW

Challenges …

… we have to face when developing a distributed system:

- Heterogeneity
- Openness
- **Security**
- Scalability
- Failure Handling
- Concurrency
- Transparency
- Quality of Service



It must be ensured that Alice and Bob are placing the order. The credit card data must not be revealed to third parties. The money transaction must not be modified by third parties.

Challenge Security

DS transmit data using public networks (e.g. Internet) → High demand for security

- Communication security: covers attacks against communication links
- Computer security: deals with detection of compromised nodes and recovery from malicious states (not covered by this lecture)

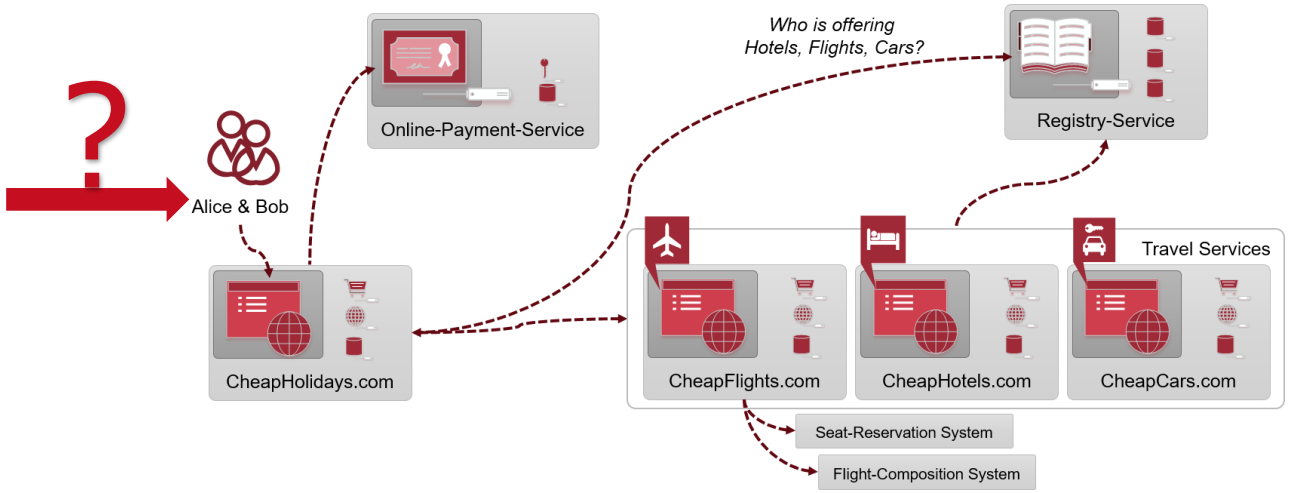
Principles of security:

- Privacy (Confidentiality, Authenticity): protection of resources against disclosure / publication to unauthorized persons, e.g. identification of external users/agents
- Integrity: protection of the resources against alteration or corruption
- Availability: protection against interference with access the resources (e.g. overload)

Challenges …

… we have to face when developing a distributed system:

- Heterogeneity
- Openness
- Security
- **Scalability**
- Failure Handling
- Concurrency
- Transparency
- Quality of Service



Alice and Bob share their great booking experience through social networks. During evening time, all notified people try to book holidays as well. CheapHolidays.com must immediately increase the number of servers to serve all the requests and avoid a crash.

Challenge Scalability

DS must work efficiently and effectively if the number of users/components increases.

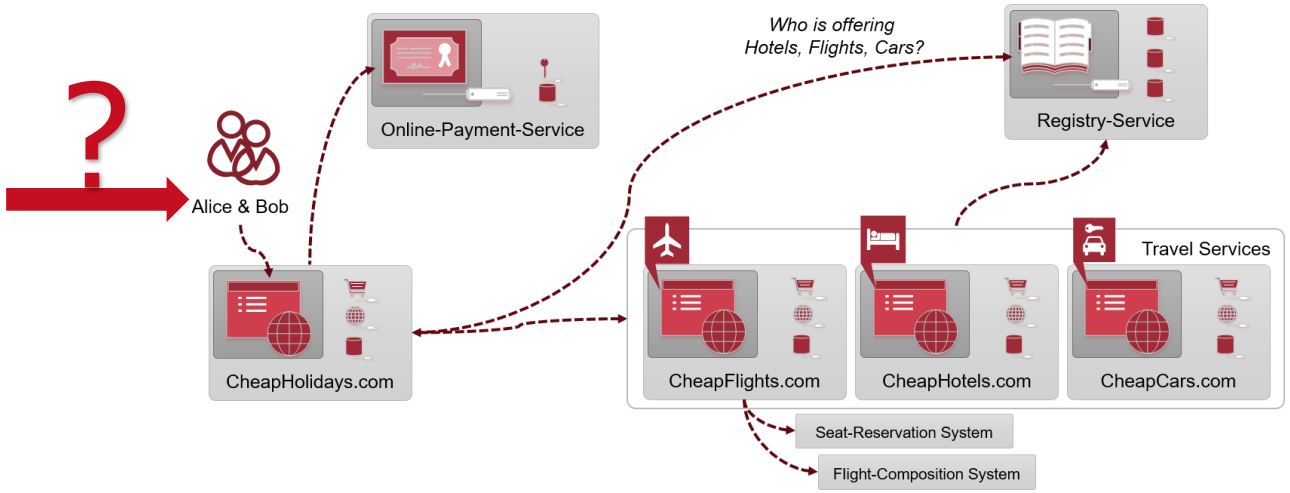
Main challenges when designing scalable distributed systems:

- Controlling cost of physical resources: with growing demand extensions possible at reasonable costs, e.g. proportional to #users/ resources (1 server for 20 users → 2 servers for 40 users)
- Controlling the performance loss: prefer hierarchical solutions
- Preventing software resources from running out: design resources in a way that they are sufficient for future extensions/expansion, e.g. IPv4 → IPv6
- Avoiding performance bottlenecks: decentralized algorithms are preferred, caching and replication techniques can be used to improve performance

Challenges …

… we have to face when developing a distributed system:

- Heterogeneity
- Openness
- Security
- Scalability
- **Failure Handling**
- Concurrency
- Transparency
- Quality of Service



The developers of CheapHolidays.com did a bad job. Alice and Bob arrive at their destination but the booked hotel has no reservations and no more free rooms. The confirmation message was lost in transit and CheapHolidays.com did not check for acknowledgements.

Challenge Failure Handling

Computer systems fail/ produce incorrect results/ stop before finished operations

DS consist of many computer systems!

- Failures in DS are partial → more difficult failure handling

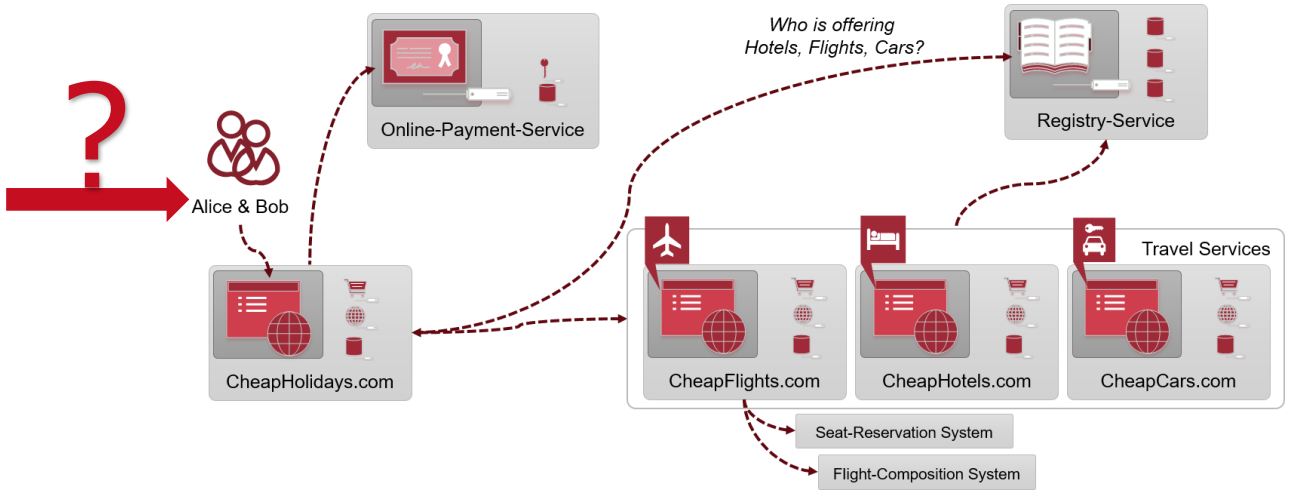
Techniques to deal with failures:

- Detecting failures: sometimes easy (e.g. checksums), in many cases failures are only guess-work (e.g. server crashed or network down)
- Masking failures: detected failures can be hidden or made less severe (e.g. by retransmission of messages, redundancy of hard disks)
- Tolerating failures: Due to amount of failures, most DS exhibit them, e.g. web site not available → browser notifies the user instead of waiting forever

Challenges …

… we have to face when developing a distributed system:

- Heterogeneity
- Openness
- Security
- Scalability
- Failure Handling
- **Concurrency**
- Transparency
- Quality of Service



Alice and Bob get divorced. To overcome their sadness, both try to book a two-week trip to Hawaii simultaneously. However, both trips comprise the same flight from CheapFlights.com and there is only one remaining seat.

Challenge Concurrency

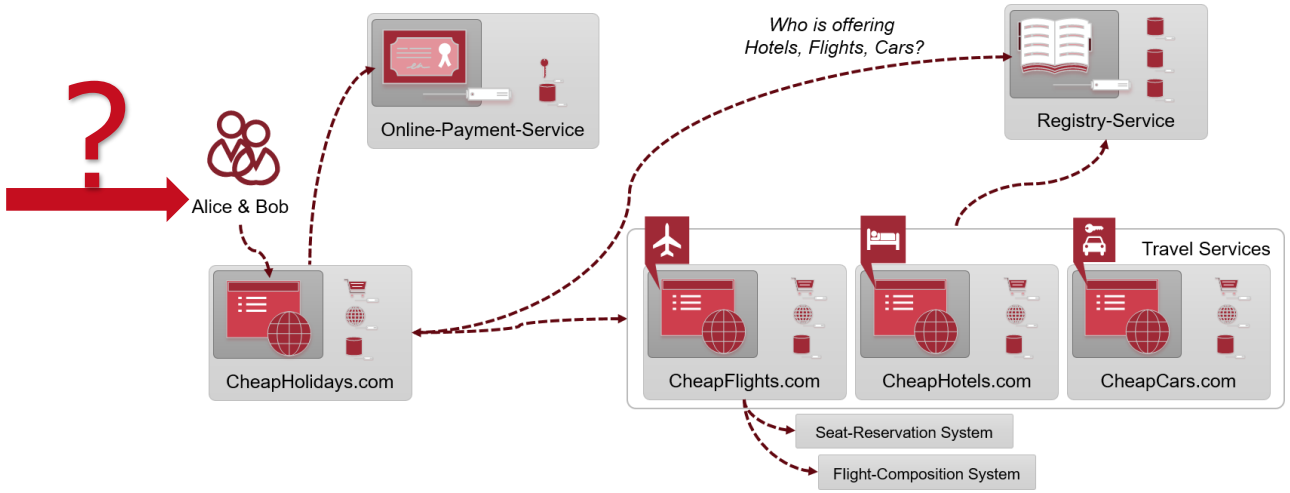
Throughput vs. consistency:

- Concurrency is essential to increase throughput
- Concurrent operations on shared resources in DS require appropriate measures to ensure consistency
- Concurrent access to shared resources must be synchronized
- Servers as well as objects in applications need to be designed to operate safe in DS
- Requires methods for transactions and distributed mutual exclusion

Challenges …

… we have to face when developing a distributed system:

- Heterogeneity
- Openness
- Security
- Scalability
- Failure Handling
- Concurrency
- **Transparency**
- Quality of Service



The booking process is complex and involves several system components. CheapHolidays hides the distribution from Alice & Bob and makes them believe that one big service is serving their needs. They perceive the distributed system as one single application.

Challenge Transparency

Transparency: concealing the separation of the components in DS from user/application

Access transparency

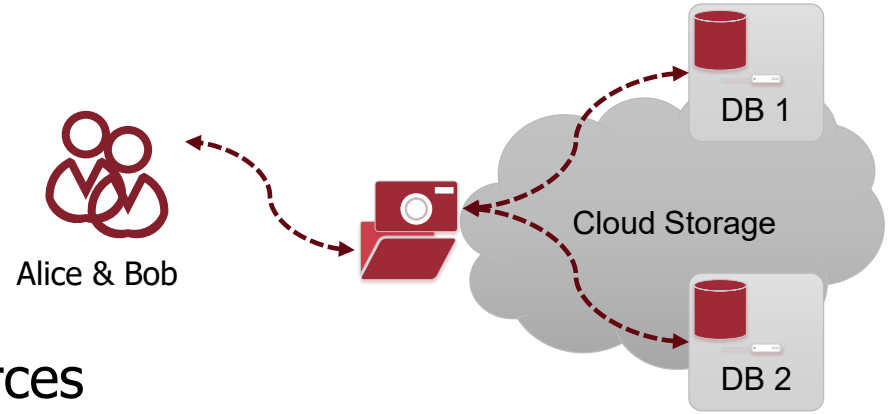
- Identical access operations for local and remote resources

Location transparency

- Resources can be accessed without knowing location

Replication transparency

- Resource replication to improve performance / reliability is invisible for the user/application



Challenge Transparency

Concurrency transparency

– Clients are masked from the effects of concurrent access to the server

Performance transparency

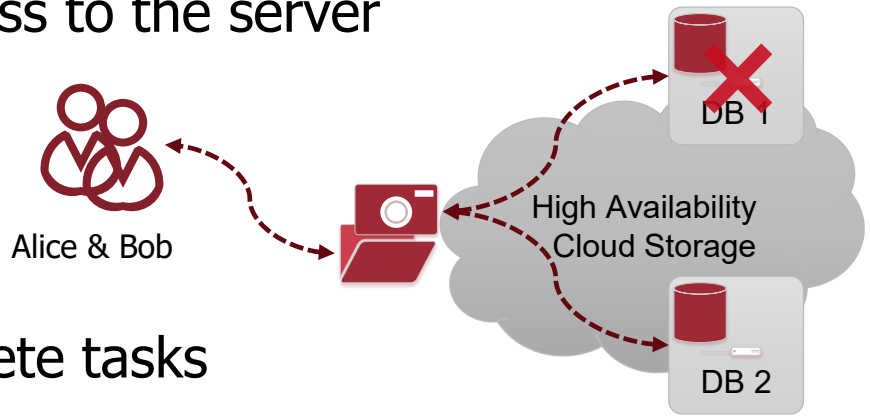
– Enables reconfiguration at varying load

Failure transparency

– Failures are masked, users and applications can complete tasks

Mobility (migration) transparency

– Moving resources do not affect the operation of the distributed system



Challenge Transparency

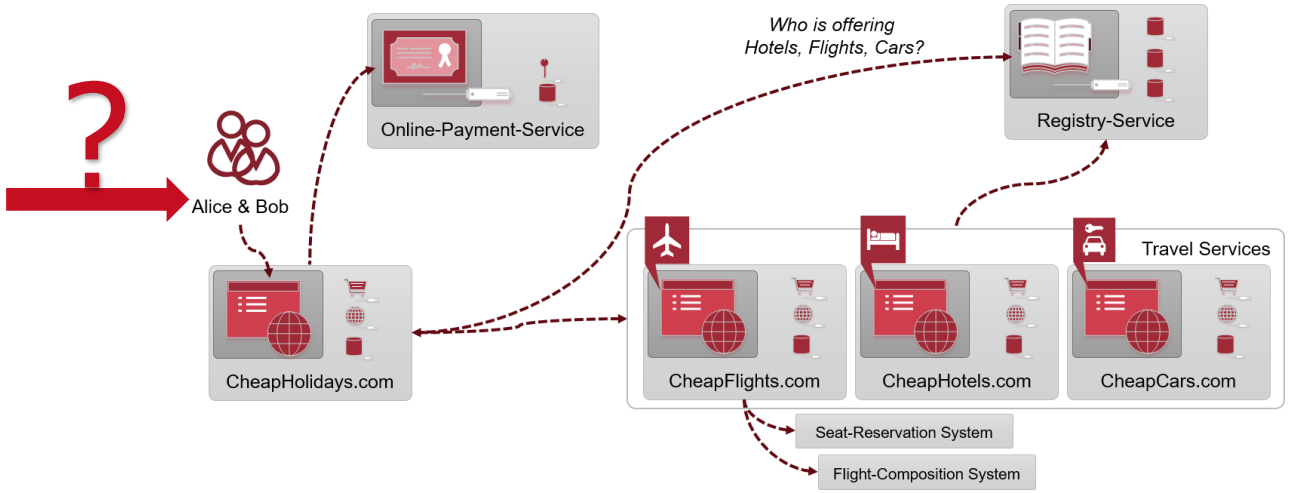
Scaling transparency: Scale out without changing the structure or algorithms

- Example: In case the cloud storage service is running out of storage capacity, additional database nodes can be plugged in instead of increasing the capacity of existing nodes.
- Speed-up vs. scaling:
  - Speed-up: amount of data and users remains, but less response time → system becomes more performant from the perspective of the user
  - (Weak) scaling: amount of data and users increase, yet response time does not increase

Challenges …

… we have to face when developing a distributed system:

- Heterogeneity
- Openness
- Security
- Scalability
- Failure Handling
- Concurrency
- Transparency
- **Quality of Service**



Search results should be offered immediately, videos of the holiday locations should be streamed in high quality, payment should not take more time than a few seconds.

Challenge Quality of Service

Non-functional properties of a system that affect Quality of Service (QoS):

- Reliability
- Security
- Adaptability: Quickly react to changes in resource availability or system configuration
  - A video source crashed, redirect requests to another stream
  - Nodes were added, redistribute users to reduce round trip times
- Performance → what we usually understand under QoS nowadays
  - The ability of a service to meet timeliness guarantees

DS offering QoS must provide resource managers that allow the application to reserve critical resources (e.g. bandwidth)

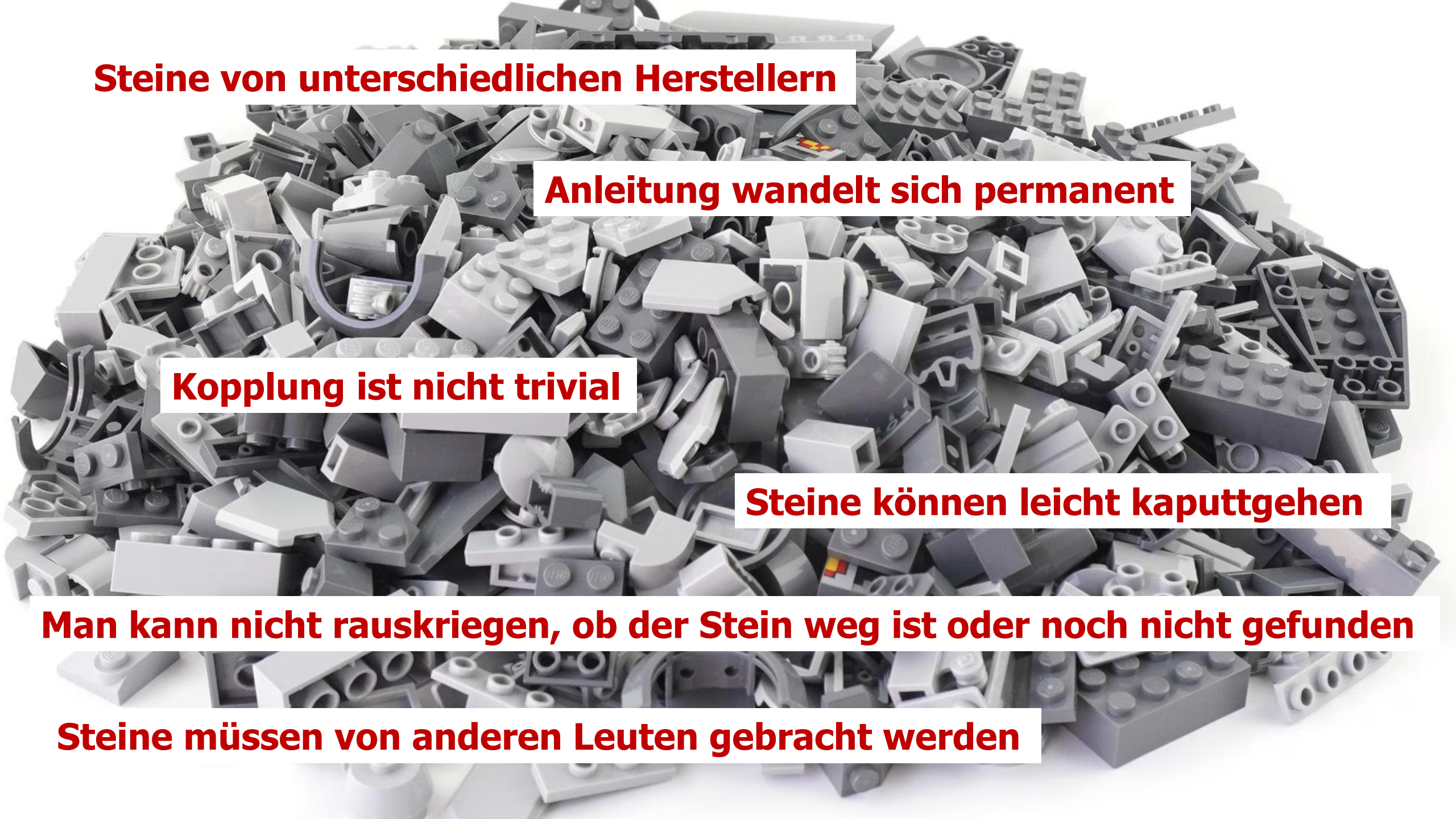
The pros and cons to keep in mind

Advantages

- Reduction of cost (scale out often cheaper than scale up)
- Availability
- Failure tolerance
- Scalability
- High performance because of parallel work
- Modular software
- Consistency with organizational structures
- Independence from producers

Disadvantages

- Complex design- und implementation process
- Problems because of heterogeneity
- Difficult verification of correctness
- Complex communication systems
- Shift from centralized to decentralized system often challenging
- Security problems
- High operation and service demand
- Difficult to estimate total cost



Steine von unterschiedlichen Herstellern

Anleitung wandelt sich permanent

Kopplung ist nicht trivial

Steine können leicht kaputtgehen

Man kann nicht rauskriegen, ob der Stein weg ist oder noch nicht gefunden

Steine müssen von anderen Leuten gebracht werden